



Universidad
Rey Juan Carlos

Aranjuez - 1 de julio de 2026

Optimización Multiobjetivo con Algoritmos Evolutivos

Fundamentos y Aspectos Prácticos

Taller 1: Optimización
Multi-Objetivo con jMetal

Antonio J. Nebro
Catedrático de Universidad



UNIVERSIDAD
DE MÁLAGA



Contenidos

1. Introducción a jMetal
2. Configuración y ejecución de algoritmos en jMetal
3. Ejercicio práctico

1

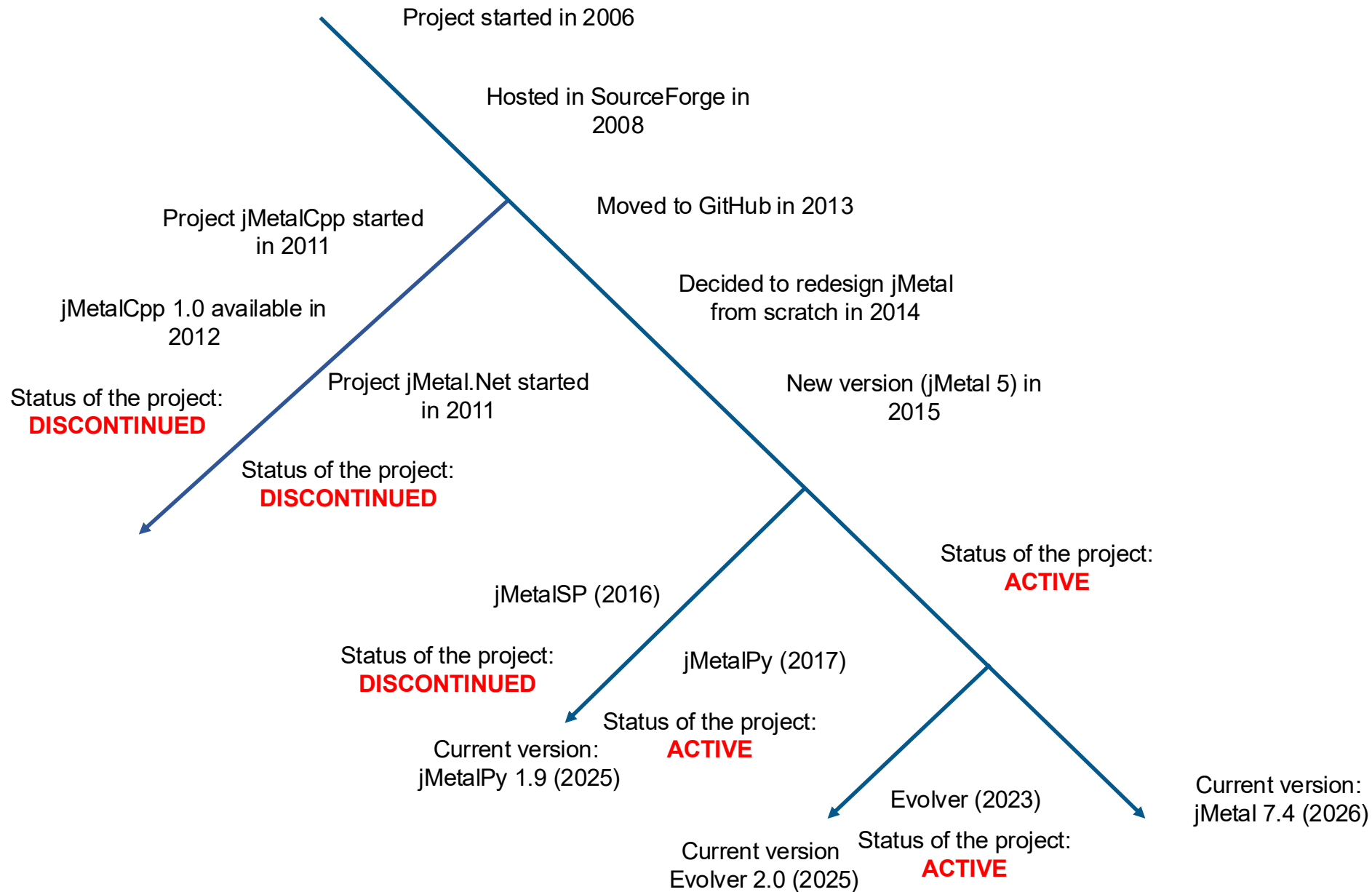
Introducción a jMetal

Motivación

- Empecé a trabajar en optimización multiobjetivo en 2003
 - Primer trabajo de investigación
 - Desarrollar un algoritmo de búsqueda dispersa (*scatter search*) multiobjetivo
 - Aplicar la experiencia adquirida durante la tesis doctoral en paralelismo a algoritmos evolutivos multiobjetivo
- Herramientas software disponibles en 2004
 - Implementación de NSGA-II en C
 - El framework PISA (también desarrollado en C)
- Problema con estas herramientas
 - Código muy difícil de comprender y de reutilizar

Motivación

- Lo que necesitaba
 - Un framework software para investigar que fuese fácil de usar, flexible, extensible y portable
 - Escrito en Java, dado que el lenguaje más popular entonces
- Solución adoptada
 - Escribir una herramienta desde cero: jMetal
 - Desarrolladores: Juan J. Durillo, Antonio J. Nebro
- Con el tiempo
 - La pusimos en dominio público
 - SourceForge (2010)
 - GitHub (2012) <https://github.com/jMetal/jMetal>



Evolución de la arquitectura de jMetal (4.5)

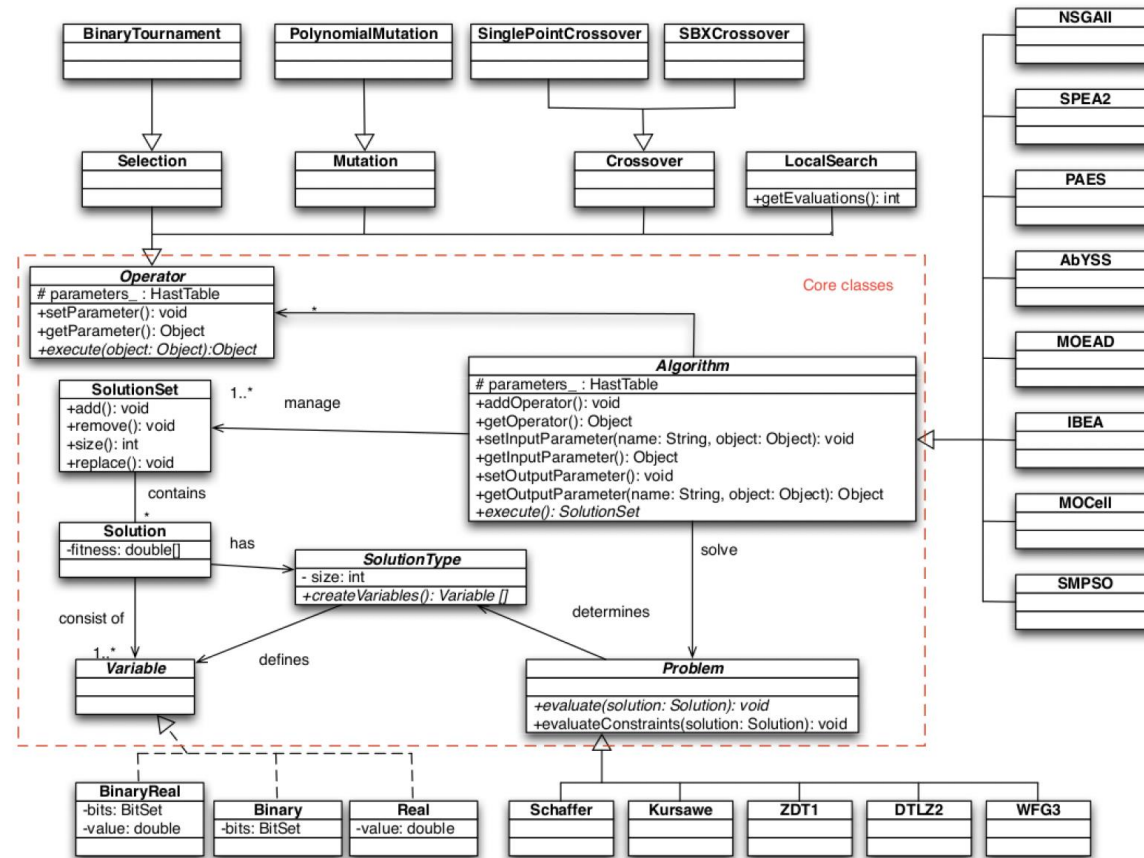
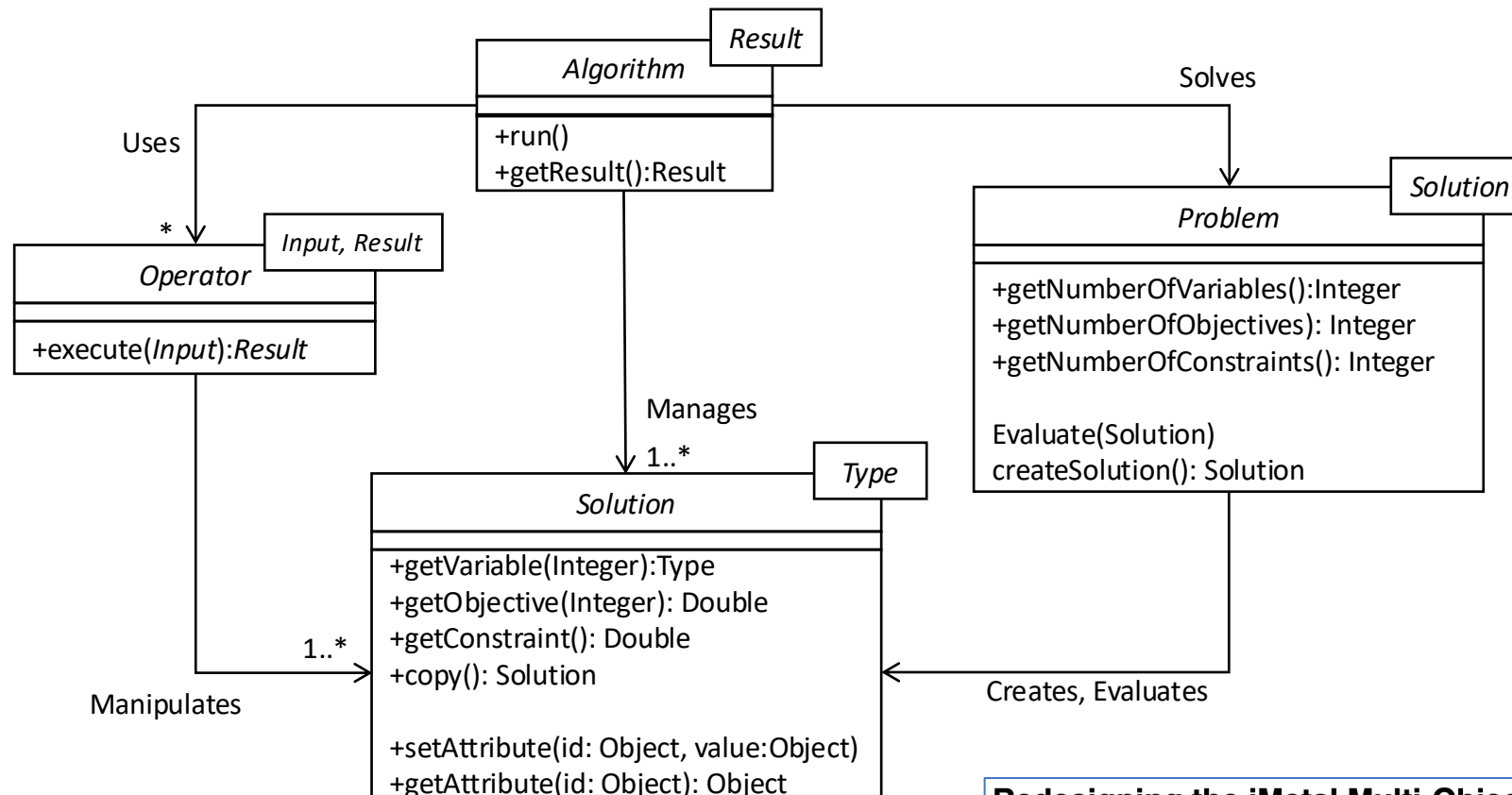


Figure 1: jMetal 4.5 architecture

<https://github.com/jMetal/jMetal/tree/v4.3>

Evolución de la arquitectura de jMetal (5.0)

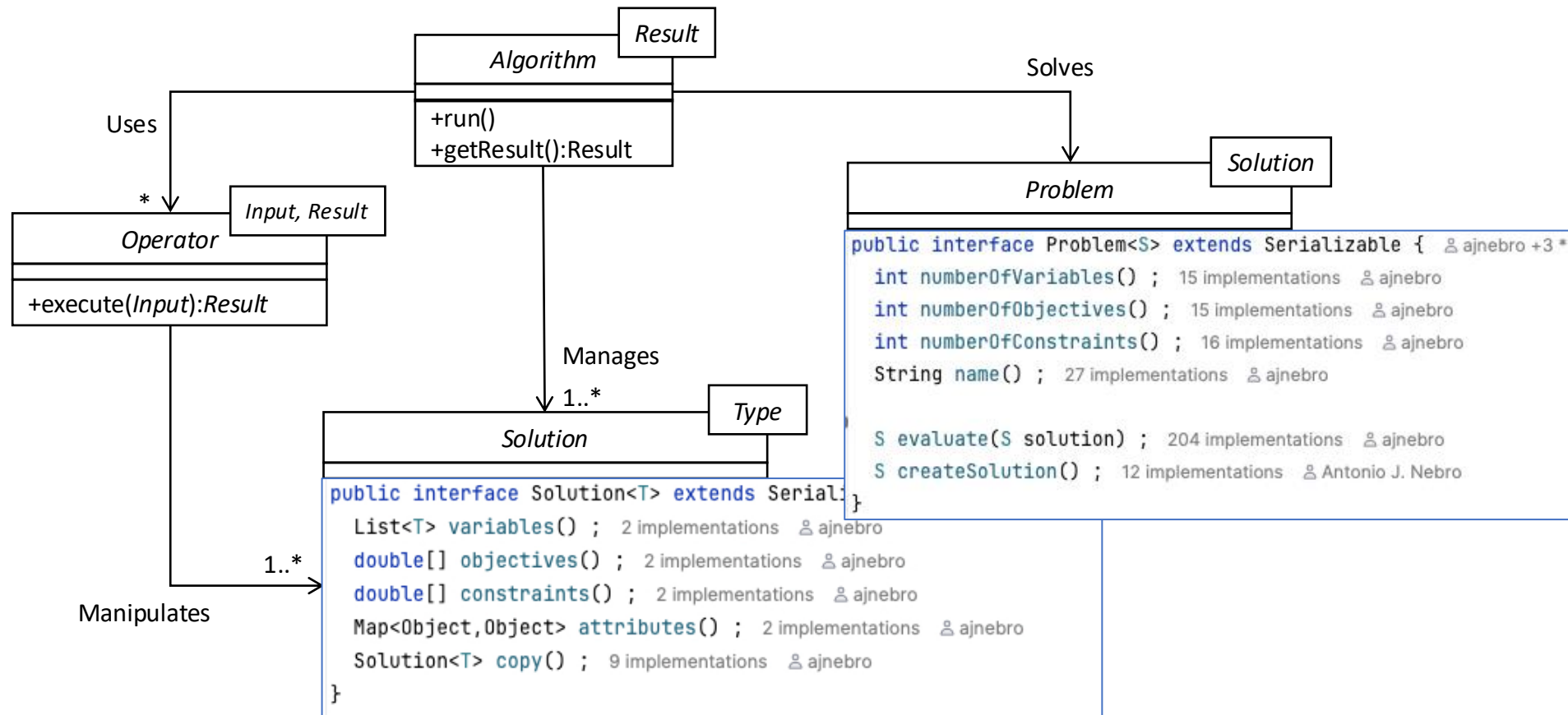


Redesigning the jMetal Multi-Objective Optimization Framework

Authors:  [Antonio J. Nebro](#),  [Juan J. Durillo](#),  [Matthieu Vergne](#) | [Authors Info & Claims](#)

GECCO Companion '15: Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation
Pages 1093 - 1100 • <https://doi.org/10.1145/2739482.2768462>

Evolución de la arquitectura de jMetal (6.0)



Diseño de algoritmos basados en componentes

- jMetal 6.0 (2022)

Algorithm 1 Pseudo-code of an evolutionary algorithm

```
1:  $P(0) \leftarrow \text{GenerateInitialSolutions}()$ 
2:  $t \leftarrow 0$ 
3: Evaluate( $P(0)$ )
4: while not StoppingCriterion() do
5:    $M(t) \leftarrow \text{Select}(P(t))$ 
6:    $Q(t) \leftarrow \text{Variate}(M(t))$ 
7:   Evaluate( $Q(t)$ )
8:    $P(t+1) \leftarrow \text{Update}(P(t), Q(t))$ 
9:    $t \leftarrow t+1$ 
10: end while
```

```
public class EvolutionaryAlgorithm<S>

    public void run() {
        population = createInitialPopulation.create();
        population = evaluation.evaluate(population);
        initProgress();
        while (!termination.isMet(attributes)) {
            List<S> matingPopulation = selection.select(population);
            List<S> offspringPopulation = variation.variate(population, matingPopulation);
            offspringPopulation = evaluation.evaluate(offspringPopulation);

            population = replacement.replace(population, offspringPopulation);
            updateProgress();
        }
    }
}
```

Component catalog

Component	Implementations
Solutions creation	Latin hypercube sampling Random Scatter Search
Evaluation	Sequential Multithreaded Sequential with external archive
Termination	Computing time Evaluations Keyboard Quality indicator
Selection	N-ary tournament Random Differential evolution Neighborhood Population and neighborhood
Variation	Crossover and mutation Differential evolution
Replacement	(μ, λ) $(\mu + \lambda)$ Ranking and density estimator Pairwise MOEA/D replacement strategy SMS-EMOA replacement strategy

Diseño de algoritmos basados en componentes

- jMetal 7.4 (2026)

jMetal 7.4 — jmetal-component catalogue

nuevo = añadido desde la versión anterior

PAQUETE	COMPONENTE	IMPLEMENTACIONES					
common	Solutions creation	Latin hypercube sampling	Random	Scatter search	Opposition-based	Sobol	Cauchy
common	Evaluation	Sequential	Multithreaded	Sequential + archive	Multithreaded + archive		
common	Termination	Evaluations	Computing time	Keyboard	Quality indicator		
ea	Selection	N-ary tournament	Random	Differential evolution	Neighborhood		
		Population and neighborhood	Truncation	Ranking	Stochastic universal sampling		
		Boltzmann					
ea	Variation	Crossover and mutation	Differential evolution				
ea	Replacement	(μ, λ)	$(\mu + \lambda)$	Ranking + density estimator	Pairwise	MOEA/D	SMS-EMOA
		Random	Tournament	Single solution	NSGA-III	RVEA	AGE-MOEA
ea	Subproblem update (nuevo)	Aggregation criterion	Feasibility rules	Violation threshold	Improved epsilon		

Diseño de algoritmos basados en componentes

- jMetal 7.4 (2026)

jMetal 7.4 — jmetal-component catalogue nuevo = añadido desde la versión anterior

PAQUETE	COMPONENTE	IMPLEMENTACIONES
common	Solutions creation	Latin hypercube sampling Random Scatter search Opposition-based Sobol Cauchy
common	Evaluation	Sequential Multithreaded Sequential + archive Multithreaded + archive
common	Termination	Evaluations Computing time Keyboard Quality indicator
ea	Selection	N-ary tournament Random Differential evolution Neighborhood Population and neighborhood Truncation Ranking Stochastic universal sampling Boltzmann
ea	Variation	Crossover and mutation Differential evolution
ea	Replacement	(μ, λ) $(\mu + \lambda)$ Ranking + density estimator Pairwise MOEA/D SMS-EMOA Random Tournament Single solution NSGA-III RVEA AGE-MOEA
ea	Subproblem update <i>(nuevo)</i>	Aggregation criterion Feasibility rules Violation threshold Improved epsilon

Diseño de algoritmos basados en componentes

- jMetal 7.4 (2026)

jMetal 7.4 — operadores de cruce y mutación por codificación

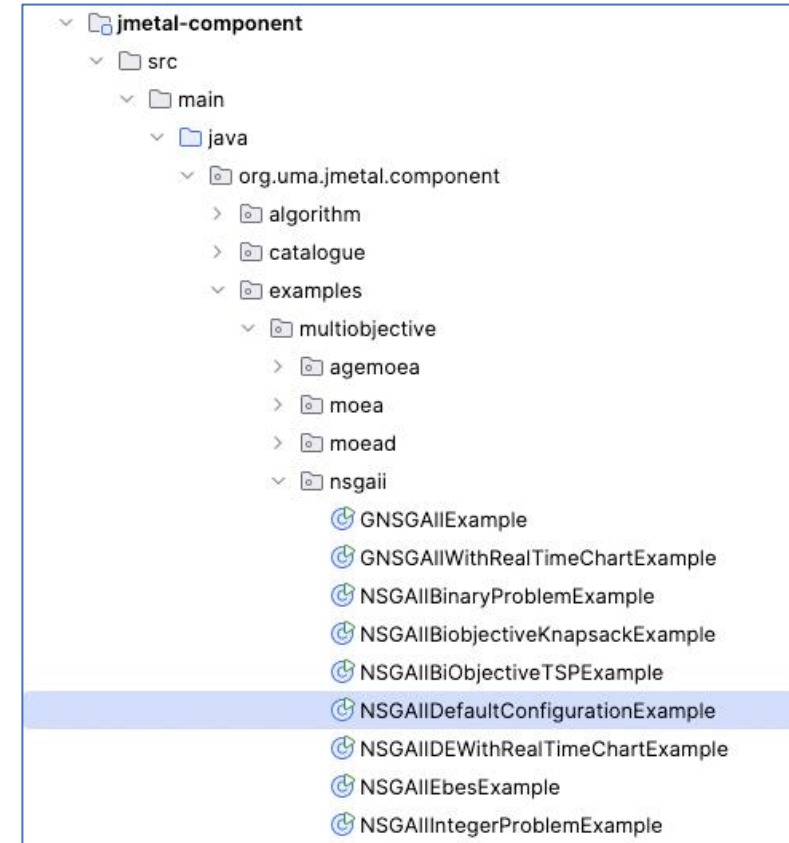
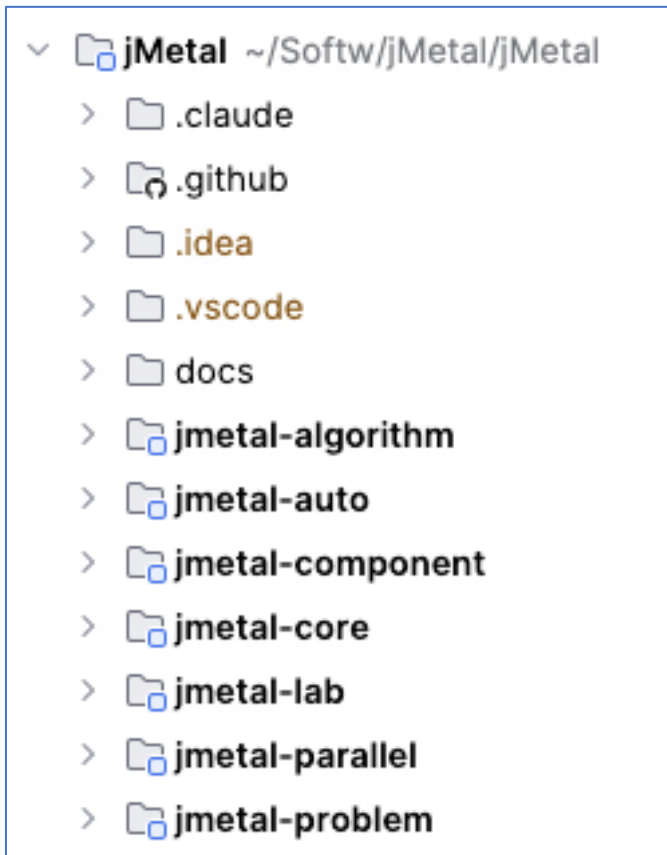
CODIFICACIÓN	CRUCE	MUTACIÓN
Real (double)	SBX Whole arithmetic Fuzzy recombination Differential evolution BLX- α BLX- α - β Laplace UNDX Arithmetic	Polynomial Simple random Non-uniform Lévy flight Uniform Power law
Entero	Integer SBX Integer polynomial	Integer polynomial Simple random
Binario	HUX Uniform Single-point Two-point N-point	Bit-flip
Permutación	PMX Edge recombination Cycle Position-based OXD	Swap Simple inversion Insert Displacement Scramble Inversion

2

Configuración y ejecución de algoritmos en jMetal

El proyecto jMetal

- jMetal es un proyecto Maven
 - Se compone de 7 subproyectos



Configuración y ejecución de NSGA-II

```
public class NSGAIIDefaultConfigurationExample { @ajnebro
    public static void main(String[] args) throws JMetalException, IOException { @ajnebro
        String problemName = "org.uma.jmetal.problem.multiobjective.zdt.ZDT4";
        String referenceParetoFront = "resources/referenceFrontsCSV/ZDT4.csv";

        Problem<DoubleSolution> problem = ProblemFactory.loadProblem(problemName);

        double crossoverProbability = 0.9;
        double crossoverDistributionIndex = 20.0;
        var crossover = new SBXCrossover(crossoverProbability, crossoverDistributionIndex);

        double mutationProbability = 1.0 / problem.numberOfVariables();
        double mutationDistributionIndex = 20.0;
        var mutation = new PolynomialMutation(mutationProbability, mutationDistributionIndex);

        int populationSize = 100;
        int offspringPopulationSize = 100;

        Termination termination = new TerminationByEvaluations( maximumNumberOfEvaluations: 25000);

        EvolutionaryAlgorithm<DoubleSolution> nsgaii =
            new NSGAIIBuilder<>(problem, populationSize, offspringPopulationSize, crossover, mutation)
                .setTermination(termination)
                .build();

        nsgaii.run();

        List<DoubleSolution> population = nsgaii.result();
        JMetalLogger.logger.info( msg: "Total execution time : " + nsgaii.totalComputingTime() + "ms");
        JMetalLogger.logger.info( msg: "Number of evaluations: " + nsgaii.numberOfEvaluations());

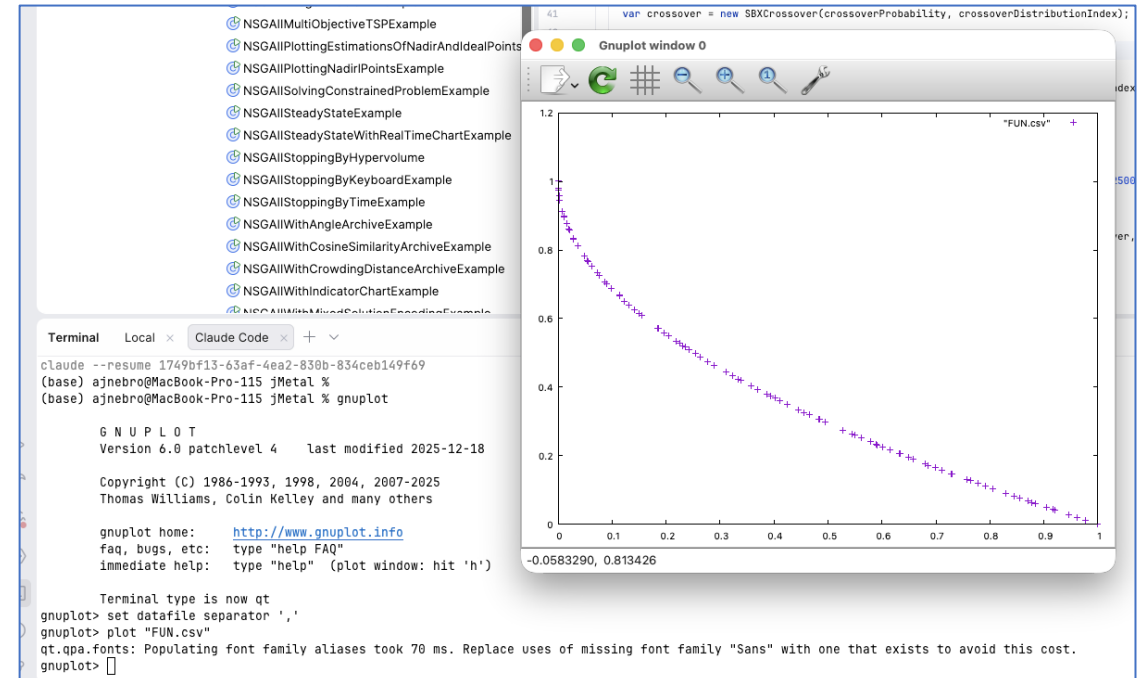
        new SolutionListOutput(population)
            .setVarFileOutputContext(new DefaultFileOutputContext( fileName: "VAR.csv", separator: ","))
            .setFunFileOutputContext(new DefaultFileOutputContext( fileName: "FUN.csv", separator: ","))
            .print();

        JMetalLogger.logger.info( msg: "Random seed: " + JMetalRandom.getInstance().getSeed());
        JMetalLogger.logger.info( msg: "Objectives values have been written to file FUN.csv");
        JMetalLogger.logger.info( msg: "Variables values have been written to file VAR.csv");

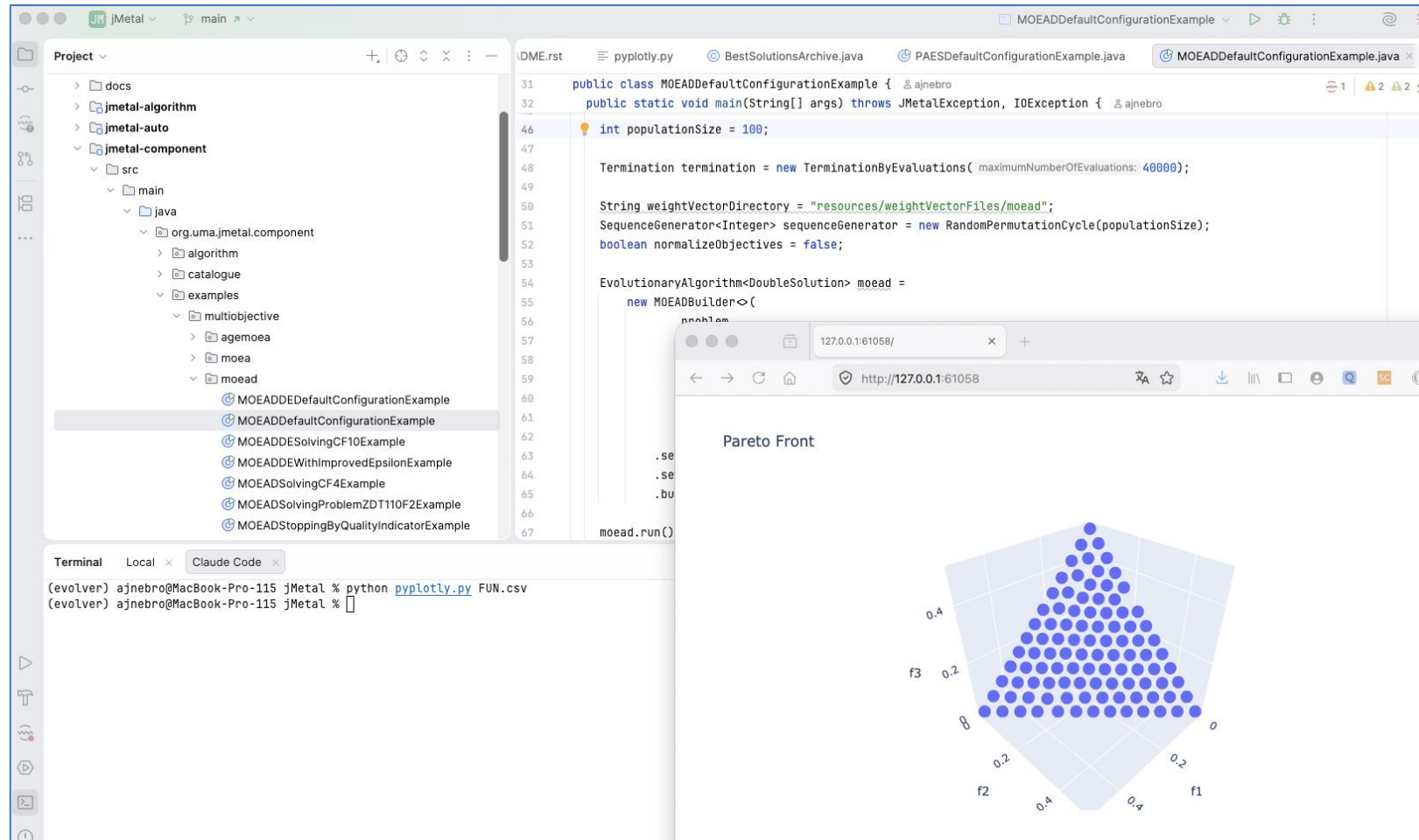
        QualityIndicatorUtils.printQualityIndicators(
            SolutionListUtils.getMatrixWithObjectiveValues(population),
            VectorUtils.readVectors(referenceParetoFront, separator: ","));
    }
}
```



```
Run NSGAIIDefaultConfigurationExample x
/Users/ajnebro/Applications/jdk-26.jdk/Contents/Home/bin/java ...
jun 25, 2026 9:38:32 A. M. org.uma.jmetal.component.examples.multiobjective.nsgaii.NSGAIIDefaultConfigurationExample main
INFO: Total execution time : 751ms
jun 25, 2026 9:38:32 A. M. org.uma.jmetal.component.examples.multiobjective.nsgaii.NSGAIIDefaultConfigurationExample main
INFO: Number of evaluations: 25000
jun 25, 2026 9:38:32 A. M. org.uma.jmetal.component.examples.multiobjective.nsgaii.NSGAIIDefaultConfigurationExample main
INFO: Random seed: 1782373111586
jun 25, 2026 9:38:32 A. M. org.uma.jmetal.component.examples.multiobjective.nsgaii.NSGAIIDefaultConfigurationExample main
INFO: Objectives values have been written to file FUN.csv
jun 25, 2026 9:38:32 A. M. org.uma.jmetal.component.examples.multiobjective.nsgaii.NSGAIIDefaultConfigurationExample main
INFO: Variables values have been written to file VAR.csv
```



Ejemplo de frente de MOEA/D (DTLZ1)



3

Ejercicio práctico

Ejercicio práctico

- Objetivo

- Evaluar algoritmos evolutivos multiobjetivo sobre problemas continuos
- Lo que se busca no es responder a "¿cuál es el mejor algoritmo?" sino "¿mejor según qué, en qué condiciones, y con qué evidencia?"

- Tareas a realizar

- **Ejecutar** — Lanzar NSGA-II, SMSEMOA, MOEA/D y PAES sobre los benchmarks ZDT, DTLZ y LZ09 con los mismos parámetros y varias semillas
- **Medir** — Calcular Hypervolume (HV) y Epsilon indicator (EP) y registrar media y desviación típica en la hoja de trabajo
- **Comparar** — Analizar qué algoritmo funciona mejor, en qué problemas, con qué indicador, y por qué los resultados no siempre cuentan la misma historia

Ejercicio práctico

- Configuraciones comunes
 - Número de soluciones a encontrar: 100
 - Condición de parada (evaluaciones)
 - ZDT: 20000
 - DTLZ: 40000
 - LZ09: 175000

Ejercicio práctico

Taller de jMetal — hoja de trabajo

Optimización multiobjetivo con algoritmos evolutivos · Registra media y desviación típica sobre ≥ 5 ejecuciones

Referencia de problemas

Problema	Tipo	Características principales
ZDT1	2D · fácil	Frente convexo. Variables sin dependencia. Punto de partida canónico.
ZDT2	2D · fácil	Frente no convexo. Prueba la presión selectiva del algoritmo.
ZDT3	2D · fácil	Frente discontinuo (5 segmentos). Pone a prueba la diversidad.
ZDT4	2D · difícil	Multimodal: 21º frentes locales. Alta dificultad de convergencia.
DTLZ1	3D	Frente lineal. Muchos óptimos locales. 3 objetivos.
DTLZ2	3D	Frente esférico. Referencia para 3 objetivos. Difícil de cubrir uniformemente.
DTLZ3	3D	Frente esférico con muchos frentes locales. Versión dura de DTLZ2.
LZ09F1/F2/F4	2D · opcional	Variables con dependencias complejas (linkage). Muy difíciles de resolver

Ejercicio práctico

Resultados — Hypervolume (HV) ↑ mayor es mejor

HV es el volumen de los puntos de un frente respecto a un punto de referencia

Problema	NSGA-II		SMSEMOA		MOEA/D		PAES		Mejor
	media	σ	media	σ	media	σ	media	σ	
2D — problemas fáciles									
ZDT1									
ZDT2									
ZDT3									
2D — difícil									
ZDT4									
3D									
DTLZ1									
DTLZ2									
DTLZ3									
2D — complejos (opcional)									
LZ09F1									
LZ09F2									
LZ09F4									

Ejercicio práctico

Resultados — Epsilon (EP) ↓ menor es mejor

EP mide el factor de escala mínimo necesario para que el frente obtenido domine al frente de referencia.

Problema	NSGA-II		SMSEMOA		MOEA/D		PAES		Mejor
	media	σ	media	σ	media	σ	media	σ	
2D — problemas fáciles									
ZDT1									
ZDT2									
ZDT3									
2D — difícil									
ZDT4									
3D									
DTLZ1									
DTLZ2									
DTLZ3									
2D — complejos (opcional)									
LZ09F1									
LZ09F2									
LZ09F4									

Ejercicio práctico

Preguntas de investigación

1. Los indicadores

- ¿HV y EP producen el mismo ranking entre algoritmos, o hay casos en que se contradicen?
- Si se contradicen, ¿cuál confiarías más para tomar una decisión y por qué?
- ¿Qué aspecto del frente captura cada indicador que el otro no captura?

2. Los algoritmos

- SMSEMOA usa HV como criterio de selección interno. ¿Obtiene siempre mejor HV en todos los problemas?
- ¿Cómo se comporta PAES frente a los otros tres? ¿A qué atribuyes la diferencia siendo trayectorial?

3. La dificultad

- ¿Qué diferencia estructural entre ZDT1 y ZDT4 explica el salto de dificultad? ¿Se refleja igual en HV y EP?
- Al pasar de ZDT a DTLZ (3 objetivos), ¿qué cambia en el problema que hace escalar peor a ciertos algoritmos?
- ¿Tiene sentido comparar numéricamente el HV de un problema 2D con el de uno 3D?

4. La variabilidad

- ¿Qué algoritmo presenta una mayor σ ?
- ¿Esa variabilidad es un problema práctico o solo estadístico?
- Si un algoritmo tiene mejor media pero mayor σ que otro, ¿cuál elegirías para un uso real?

5. Conexión con vuestra investigación

- ¿Qué propiedades de ZDT4 o DTLZ3 tienen un análogo en los problemas combinatorios que trabajáis?
- Si adaptaseis NSGA-II a vuestro problema combinatorio, ¿qué componente cambiaríais primero y por qué?
- ¿Qué aportaría usar HV o EP frente al ratio de soluciones no dominadas o la distancia al óptimo?

6. Reflexión final

- Con los datos de la tabla, ¿se puede afirmar que un algoritmo es mejor que otro, o la respuesta depende siempre del problema?
- ¿Qué experimento adicional haríais si tuvierais más tiempo para llegar a una conclusión más sólida?
- ¿Qué limitación de este estudio señalaríais si fuerais a publicar estos resultados?

Observaciones y conclusiones

Observaciones y conclusiones: